

```

package repository.search;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;

import config.DatabaseConnector;
import dto.search.SearchDto;

/**
 * 検索機能用 Repository
 */
public class SearchRepository {

    /**
     * キーワードが入力されている場合、検索候補を最大10件取得する。
     * 優先順位は ユーザー → チャンネル → メッセージ の順。
     *
     * @param keyword 検索キーワード
     * @param userId ログインユーザーID
     * @return 検索候補 List
     * @throws SQLException SQL 例外発生時
     */
    public List<SearchDto> find(String keyword, String userId) throws SQLException {

        List<SearchDto> list = new ArrayList<>();

        String sql =
            "SELECT type, word FROM ( " +

            // 自分と同じワークスペース内のユーザーを検索
            "SELECT DISTINCT 'USER' AS type, u.display_name AS word, 1 AS priority " +
            "FROM slick_user u " +
            "JOIN slick_user me ON u.workspace_id = me.workspace_id " +
            "WHERE me.id = ? " +
            "AND (u.name LIKE ? ESCAPE '\\' " +
            "OR u.display_name LIKE ? ESCAPE '\\') " +

```

```
"UNION ALL " +
```

```
// 自分が参加しているチャンネルを検索
```

```
"SELECT 'CHANNEL' AS type, c.channel_name AS word, 2 AS priority " +
```

```
"FROM channel c " +
```

```
"JOIN user_channel uc ON c.channel_id = uc.channel_id " +
```

```
"WHERE uc.id = ? " +
```

```
"AND c.channel_name LIKE ? ESCAPE '\\' " +
```

```
"UNION ALL " +
```

```
// 自分が参加しているチャンネルのメッセージを検索
```

```
"SELECT 'MESSAGE' AS type, m.message_text AS word, 3 AS priority " +
```

```
"FROM message m " +
```

```
"JOIN user_channel uc2 ON m.channel_id = uc2.channel_id " +
```

```
"WHERE uc2.id = ? " +
```

```
"AND m.message_text LIKE ? ESCAPE '\\' " +
```

```
") " +
```

```
"ORDER BY priority " +
```

```
"FETCH FIRST 10 ROWS ONLY";
```

```
try (
```

```
    Connection conn = DatabaseConnector.getConnection();
```

```
    PreparedStatement ps = conn.prepareStatement(sql)
```

```
) {
```

```
    String word = "%" + escapeLike(keyword) + "%";
```

```
// ユーザー検索用
```

```
ps.setString(1, userId);
```

```
ps.setString(2, word);
```

```
ps.setString(3, word);
```

```
// チャンネル検索用
```

```
ps.setString(4, userId);
```

```
ps.setString(5, word);
```

```
// メッセージ検索用
```

```
ps.setString(6, userId);
```

```

        ps.setString(7, word);

        try (ResultSet rs = ps.executeQuery()) {

            while (rs.next()) {

                list.add(new SearchDto(
                    rs.getString("type"),
                    rs.getString("word")
                ));
            }
        }

        return list;
    }

    /**
     * 検索欄が空の場合、検索履歴を最大10件取得する。
     *
     * @param userId ログインユーザーID
     * @return 検索履歴 List
     * @throws SQLException SQL 例外発生時
     */
    public List<SearchDto> findHistory(String userId) throws SQLException {

        List<SearchDto> list = new ArrayList<>();

        String sql =
            "SELECT 'HISTORY' AS type, search_history AS word " +
            "FROM search " +
            "WHERE id = ? " +
            "ORDER BY search_id DESC " +
            "FETCH FIRST 10 ROWS ONLY";

        try (
            Connection conn = DatabaseConnector.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)
        ) {

            ps.setString(1, userId);

```

```

try (ResultSet rs = ps.executeQuery()) {

    while (rs.next()) {

        list.add(new SearchDto(
            rs.getString("type"),
            rs.getString("word")
        ));
    }
}

return list;
}

/**
 * 検索履歴を登録する。
 *
 * @param userId ログインユーザーID
 * @param keyword 検索キーワード
 * @throws SQLException SQL 例外発生時
 */
public void insertHistory(String userId, String keyword) throws SQLException {

    String sql =
        "INSERT INTO search(id, search_history) " +
        "VALUES (?, ?)";

    try (
        Connection conn = DatabaseConnector.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)
    ) {

        ps.setString(1, userId);
        ps.setString(2, keyword);

        int ret = ps.executeUpdate();

        if (ret == 0) {
            throw new SQLException("検索履歴の登録に失敗しました。");
        }
    }
}

```

```
    }  
  }  
}  
  
/**  
 * LIKE 検索用のエスケープ処理。  
 *  
 * % と _ は LIKE 検索で特別な意味を持つため、  
 * 普通の文字として扱えるようにする。  
 *  
 * @param keyword 検索キーワード  
 * @return エスケープ後の文字列  
 */  
private String escapeLike(String keyword) {  
  
    if (keyword == null) {  
        return "";  
    }  
  
    return keyword  
        .replace("\\", "\\\\")  
        .replace("%", "\\%")  
        .replace("_", "\\_");  
}  
}
```